

Bahasa Pemrograman C++ untuk SMA/MA

DASAR PEMROGRAMAN C++

Contact :

Email :

brotherbangun@gmail.com

Telp. : 0823 2773 9040

IG : @bangun_www

Youtube : @bangun_www

Office :

MAN Purworejo, Jl. Hengloend Katamso

No. 51 Juru Tengah, Purworejo, 54114

Address :

Perum Green Pangen No. B1,

Pangenrejo, Purworejo, Jawa Tengah,

TABLE OF CONTENTS

A. STRUKTUR DASAR_____1

1. Header_____
2. Fungsi Main_____
3. Variabel_____
4. Statement_____
5. Struktur Kendali_____
6. Fungsi_____
7. Komentar_____

B. TIPE DATA

1. Primitive (Sederhana)
2. Non-Primitive (Kompleks)

C. VARIABEL

1. Konsep Variabel
2. Aturan Penamaan Variabel

D. OPERATOR

1. Operator Aritmatika
2. Operator Penugasan
3. Operator Perbandingan
4. Operator Logika
5. Operator Increment dan Decrement

E. PERCABANGAN

1. IF
2. IF ELSE
3. IF ELSE IF
4. SWITCH

F. PERULANGAN

1. FOR
2. WHILE
3. DO-WHILE

G. FUNGSI

1. Fungsi Standar
2. Fungsi Pengguna
3. Parameter dan Argumen dalam Fungsi
4. Return dalam Fungsi
5. Fungsi Rekursif

C++

C++ adalah bahasa pemrograman umum yang sering digunakan untuk membangun berbagai macam perangkat lunak.

A. STRUKTUR DASAR

1. Header

Program C++ dimulai dengan header yang biasanya digunakan untuk menyertakan file header standar yang diperlukan untuk program.

Contoh :

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
```

2. Fungsi Main (Fungsi Utama)

Setiap program C++ harus memiliki fungsi utama (main function) yaitu fungsi yang akan dieksekusi pertama kali saat program dijalankan. **Di dalam fungsi main inilah kita akan menuliskan kode program.**

Contoh :

```
5  int main() {
6      // kode program diketik disini
7      return 0;
8  }
```

3. Variabel

Variabel digunakan untuk **menyimpan data** dalam program. Variabel **harus** dideklarasikan/**dibuat terlebih dahulu** sebelum digunakan, dengan cara menuliskan **tipe data** (spasi) **nama variabel** nya. Contoh berikut kita membuat variabel di dalam fungsi main. **Tipe data** dibahas di halaman 4.

Contoh :

a. Deklarasi variabel yang belum ada isinya.

```
5  int main() {
6      int angka;
7      float nilai;
8      char karakter;
9      return 0;
10 }
```

membuat 3 buah variabel yang yang belum ada isi datanya, dengan tipe data berbeda-beda

b. Deklarasi variabel yang diberi nilai awal

```

5  int main() {
6      int angka = 10;
7      float nilai = 3.14;
8      char karakter = 'a';
9      return 0;
10 }
```

membuat 3 buah variabel dengan tipe data berbeda-beda dan langsung diberi nilai awal / diberi isian data

c. Deklarasi variabel konstanta / Variabel yang nilainya tetap

```

5  int main() {
6      const float PI = 3.14;
7      const int JUMLAH_KAKI = 4;
8      return 0;
9  }
```

membuat 2 buah variabel yang nilai / isi datanya selalu tetap

d. Deklarasi variabel dengan cara Multiple Declaration

```

5  int main() {
6      int angka1, angka2, angka3;
7      float nilai1, nilai2, nilai3;
8      return 0;
9  }
```

membuat 6 buah variabel dimana 3 variabel bertipe data int dan 3 variabel yang lain

e. Deklarasi variabel dengan tipe data Array

```

5  int main() {
6      int arr[5];
7      float nilai[3] = {3.14, 2.71, 1.62};
8      char huruf[5] = {'a', 'b', 'c', 'd', 'e'};
9      return 0;
10 }
```

4. Statement

Statement atau pernyataan digunakan untuk melakukan **tindakan** tertentu dalam program, seperti pengambilan **input (cin)**, **perhitungan** matematika, atau **output (cout)** ke layar.

```

5  int main() {
6      int sisi, luas;
7
8      cout << "Program Hitung Luas Persegi:" << endl;
9      cout << "===== " << endl;
10     cout << "Masukkan data sisi = ";
11     cin >> sisi;
12     luas = sisi * sisi;
13     cout << "Luas Persegi = " << luas << endl;
14
15     return 0;
16 }
```

Tampilan program ketika dijalankan dan sudah diinputkan nilai untuk variabel sisi :

```
Program Hitung Luas Persegi:
=====
Masukkan data sisi = 10
Luas Persegi = 100
```

5. Struktur Kendali

Struktur kendali digunakan untuk mengontrol alur program, seperti **percabangan (hal. 7)** dan **perulangan (hal. 9)**.

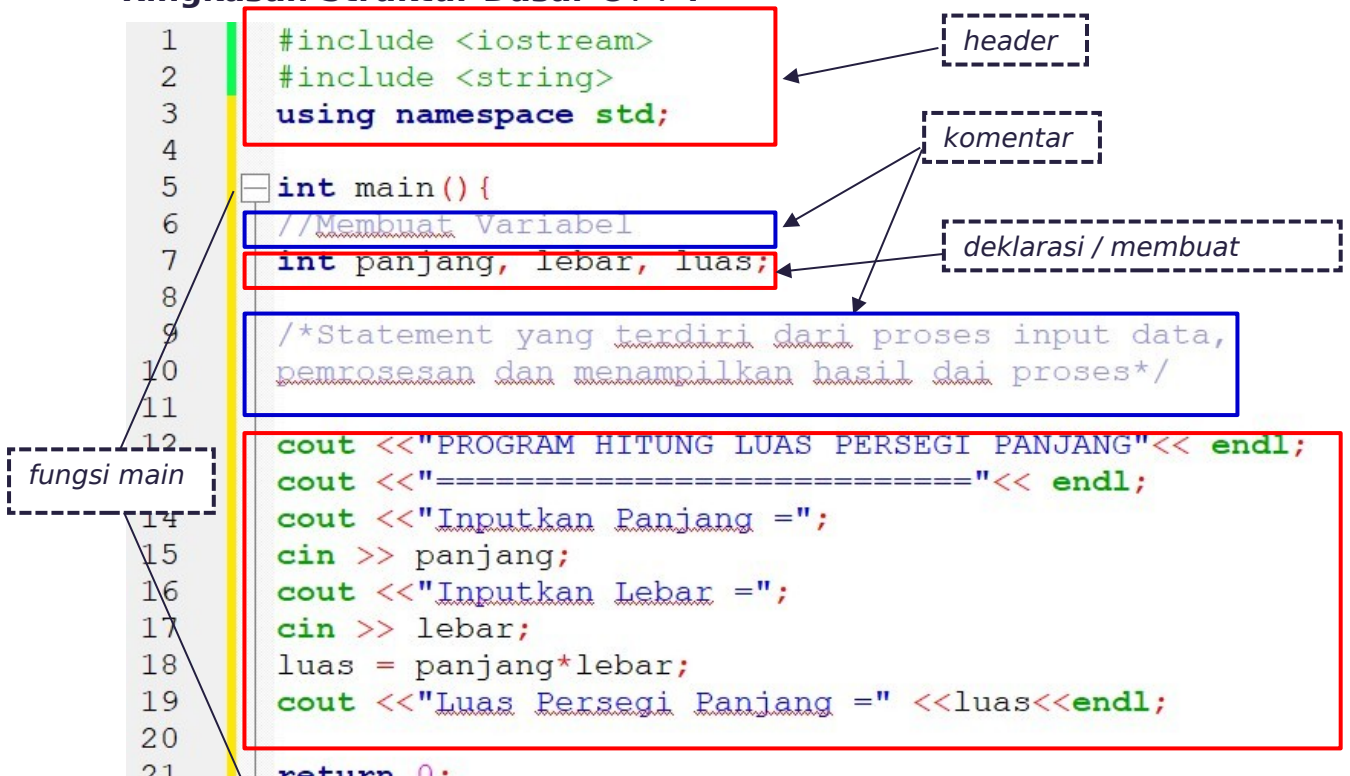
6. Fungsi

Fungsi adalah blok kode yang dapat dipanggil dari tempat lain dalam program, dan dapat menerima argumen dan mengembalikan nilai.

7. Komentar

Komentar digunakan untuk **memberikan penjelasan** pada kode program, dan **tidak** akan dieksekusi oleh program. Menggunakan simbol `//` untuk komentar dalam 1 baris. Dan menggunakan simbol `/* komentar */` untuk komentar yang lebih dari 1 baris.

Ringkasan Struktur Dasar C++ :





B. TIPE DATA

1. Primitive (Sederhana)

a. Integer ditulis **int**

Tipe data untuk angka bulat. *Contoh* : 10, 9, atau 17.

b. Float ditulis **float**

Tipe data untuk angka pecahan. *Contoh* : 3.14, 2.55, atau 10.5 .

c. Boolean ditulis **bool**

Tipe data yang berisi nilai **true** atau **false**.

d. Char ditulis **char**

Tipe data untuk 1 karakter. *Contoh* : 'A', 'B' atau '\$'.

2. Non-Primitive (Kompleks)

a. String ditulis **string**

Tipe data untuk kumpulan karakter. *Contoh* : "Saya siswa MAN Purworejo"

b. Array

Tipe data untuk kumpulan data yang sejenis. Cara penulisan : **tipe_data_primitive** (spasi) **nama_variabel**[ukuran]. Index dari Array dihitung mulai dari index ke-0.

Contoh :

▪ **string>NamaSiswa[6] = {"andi", "agus", "budi", "bayu", "caca", "cucu"}**

Artinya : membuat variabel dengan nama **NamaSiswa** yang berupa array dan merupakan kumpulan dari data string (kumpulan karakter), terdiri dari 6 buah data. Kata "*dzaki*" berada pada index ke-3

▪ **int nilai [5] = {3, 5, 8, 10, 11}**

Artinya : membuat variabel dengan nama **nilai** yang berupa array dan merupakan kumpulan dari data integer (angka bulat), terdiri dari 5 buah data.

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int    variabel_1 = 100;
8      float  variabel_2 = 10.5;
9      bool   variabel_3 = true;
10     char   variabel_4 = 'A';
11     string  variabel_5 = "Saya Siswa MAN Purworejo";
12     int     variabel_6[5] = {10, 20, 40, 54, 73};
13     string  variabel_7[3] = {"jihan", "naufal", "syafa"};
14     return 0;
15 }

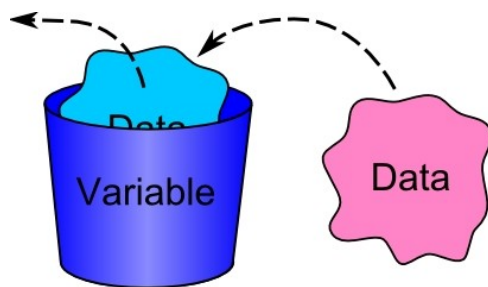
```

penulisan tipe data dan pendeklarasian variabel

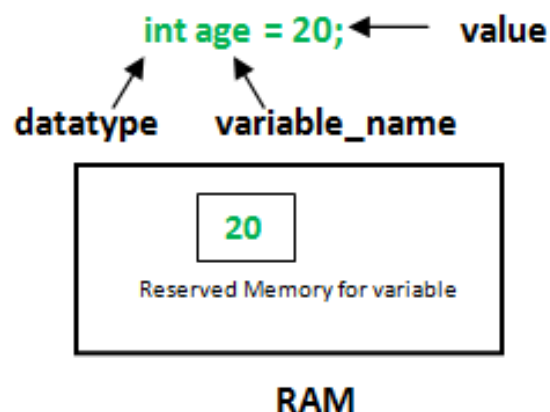
5

C. VARIABEL

1. Konsep Variabel



Sudah dibahas sebelumnya bahwa sesuai ilustrasi disamping, variabel diibaratkan sebuah **tempat penyimpanan**. Nilainya bisa ditentukan diawal atau bisa berubah sepanjang kode program sesuai inputan data dari pengguna.



2. Aturan Penamaan Variabel :

- Variabel bisa terdiri dari **huruf**, **angka** dan karakter **underscore** (`_`).
- Karakter pertama** dari variabel **hanya** boleh berupa **huruf** dan **underscore** (`_`), tidak bisa berupa angka.

- c. Variabel harus selain dari keyword. Contoh, kita tidak bisa memakai kata **int** sebagai nama variabel, karena int merupakan keyword untuk menandakan tipe data integer. Contoh lain lagi misalnya **for**, **while**, **if**.

D. OPERATOR

1. Operator Aritmatika

Operator aritmatika digunakan untuk melakukan operasi matematika pada dua atau lebih variabel. Contohnya:

```
6      int a = 10, b = 5;
7      int c = a + b; // penjumlahan
8      int d = a - b; // pengurangan
9      int e = a * b; // perkalian
10     int f = a / b; // pembagian
11     int g = a % b; // modulus sisa pembagian
```

2. Operator Penugasan

Operator penugasan digunakan untuk memberikan nilai pada variabel. Contohnya:

```
6      int x = 20;
7      int y = 10;
8      x += y; // sama dengan x = x + y;
9      y -= x; // sama dengan y = y - x;
```

3. Operator Perbandingan

Operator perbandingan digunakan untuk membandingkan dua nilai dan menghasilkan nilai boolean (true atau false). Contohnya:

```
6      int a = 20, b = 10;
7      bool c = a > b; // true, karena a lebih besar dari b
8      bool d = a == b; // false, karena a tidak sama dengan b
9      bool e = a != b; // true, karena a tidak sama dengan b
```

4. Operator Logika

Operator perbandingan digunakan untuk membandingkan dua nilai dan menghasilkan nilai boolean (true atau false). Contohnya:

```
6      bool a = true, b = false;
7      bool c = a && b; // false, karena a dan b adalah false
8      bool d = a || b; // true, karena a adalah true
9      bool e = !a; // false, karena a adalah true
```

5. Operator Increment dan Decrement

Operator increment dan decrement digunakan untuk menambah atau mengurangi nilai variabel sebesar satu. Contohnya:

```
6      int a = 10;  
7      a++; // sama dengan a = a + 1;  
8      a--; // sama dengan a = a - 1;
```

Highlight

Operat or	Nama	Penjelasan	Contoh
&&	And	Akan menghasilkan true jika kedua operand true	true && false, hasilnya: false
	Or	Akan menghasilkan true jika salah satu operand true	true false, hasilnya: true
!	Not	Akan menghasilkan true jika operand false	!false, hasilnya: true
%	Modulo	Sisa hasil bagi	
a++	Increment	a = a + 1 sampai kondisi terpenuhi	
a --	Decrement	a = a - 1 sampai kondisi terpenuhi	
+=	Penambah an langsung	a += b artinya a = a + b	

E PERCABANGAN

Dalam pembuatan program, ada saatnya kita butuh suatu percabangan, yakni jika sebuah kondisi terpenuhi, jalankan kode program yang dipilih, jika tidak terpenuhi jalankan kode program yang lain. Struktur percabangan dalam bahasa pemrograman C++ terdiri dari beberapa jenis, yaitu **if**, **if else**, **if else if**, dan **switch**.

1. IF

Struktur **IF** digunakan untuk menjalankan sebuah blok kode jika kondisi tertentu terpenuhi. Struktur defaultnya adalah :

```
IF (condition) {  
    // statement code program  
}
```

Contoh :

```

1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      int a;
7      cout << "Input Angka Bulat : ";
8      cin >> a;
9
10     if (a % 2 == 0) {
11         cout << a << " adalah angka genap" << endl;
12     }
13     if (a % 2 == 1) {
14         cout << a << " adalah angka ganjil" << endl;
15     }
16
17     return 0;
18 }

```

blok percabangan IF

Logika percabangan IF yang pertama adalah jika **a** diinputkan dengan bilangan yang jika dibagi 2, sisa hasil baginya adalah 0, maka **a** adalah bilangan **genap**. Sehingga program akan menampilkan kalimat : **(isi dari a) adalah angka genap**. Misalnya angka 4 yang diinputkan, maka program akan menampilkan : **4 adalah angka genap**. Logika percabangan IF kedua adalah jika **a** diinputkan dengan bilangan yang jika dibagi 2, sisa hasil baginya adalah 1, maka **a** adalah bilangan **ganjil**. Sehingga program akan menampilkan kalimat : **(isi dari b) adalah angka ganjil**. Misalnya angka 5 yang diinputkan, maka program akan menampilkan : **5 adalah angka genap**.

2. IF ELSE

Struktur IF ELSE digunakan untuk menjalankan sebuah blok kode program jika **kondisi tertentu terpenuhi**, dan menjalankan blok kode lain jika kondisi tersebut tidak terpenuhi. Struktur defaultnya adalah :

```

IF (condition) {
    // true statement program code
}
ELSE {
    // false statement program code
}

```

Contoh :

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int number;
7      cout << "Masukkan sebuah bilangan : ";

```

blok percabangan IF ELSE



Logika percabangan IF ELSE pada contoh diatas adalah untuk logika **IF** di baris nomor 10, jika isi dari variabel number dibagi 2 sisa hasil baginya adalah 0, maka isi dari variabel number adalah bilangan genap. Jika kondisi tersebut tidak terpenuhi, maka program akan melanjutkan ke bagian **ELSE** dan langsung menampilkan bahwa isi dari variabel number adalah bilangan ganjil.

3. IF ELSE IF

Struktur **IF ELSE IF** digunakan untuk mengecek kondisi percabangan lebih dari dua.

```
IF (first condition) {  
    // first statement program code  
}  
ELSE IF (second condition) {  
    // second statement program code  
}  
ELSE (third condition) {  
    // third statement program code  
}
```

Contoh :

```
1  #include <iostream>  
2  using namespace std;  
3  
4  int main() {  
5      int nilai;  
6      cout << "Masukkan nilai mahasiswa: ";  
7      cin >> nilai;  
8  
9      if (nilai >= 90) {  
10         cout << "Grade A" << endl;  
11     } else if (nilai >= 80) {  
12         cout << "Grade B" << endl;  
13     } else if (nilai >= 70) {  
14         cout << "Grade C" << endl;  
15     } else if (nilai >= 60) {  
16         cout << "Grade D" << endl;  
17     } else {
```

blok percabangan IF ELSE

"r

Pada contoh logika percabangan IF ELSE IF diatas, terdapat 5 blok pilihan percabangan dimulai dari baris ke-9 sampai baris ke 19. Pilihan pertama adalah jika isi dari variabel nilai ≥ 90 , maka akan ditampilkan : **Grade A**. Jika tidak terpenuhi, maka program akan membaca percabangan kedua dimana isi dari variabel nilai akan dicek kembali apakah ≥ 80 atau tidak, jika iya maka akan ditampilkan : **Grade B**. Jika tidak terpenuhi program akan masuk ke kondisi percabangan ketiga dan seterusnya sampai percabangan terakhir (ELSE).

4. SWITCH

Struktur **SWITCH** digunakan untuk mengecek nilai sebuah variabel dan menjalankan blok kode tertentu sesuai dengan nilai tersebut. Struktur defaultnya :

```
SWITCH (expression or variabel) {  
    case value1:  
        // statement of code to be executed if expression matches value1  
        break;  
    case value2:  
        // statement of code to be executed if expression matches value2  
        break;  
    // more cases  
    default:  
        // statement of ode to be executed if expression doesn't match any of  
        the values  
}
```

Contoh :

```
1  #include <iostream>  
2  using namespace std;  
3  
4  int main() {  
5      char kodeBarang;  
6      cout << "Masukkan kode barang (A, B, C): ";  
7      cin >> kodeBarang;  
8  
9      switch (kodeBarang) {  
10         case 'A':  
11             cout << "Harga barang: Rp. 10.000,-" << endl;  
12             break;  
13         case 'B':  
14             cout << "Harga barang: Rp. 25.000,-" << endl;  
15         case 'C':  
16             cout << "Harga barang: Rp. 50.000,-" << endl;  
17         default:  
18             cout << "Kode barang tidak valid!" << endl;  
19     }
```

blok percabangan SWITCH

Logika pada contoh percabangan SWITCH diatas adalah jika variabel **kodeBarang** diisi dengan karakter A, maka program akan menampilkan : **Harga barang : Rp. 10.000,-** . jika variabel **kodeBarang** diisi dengan karakter B, maka program akan menampilkan : **Harga barang : Rp. 25.000,-**. jika variabel **kodeBarang** diisi dengan karakter C, maka program akan menampilkan : **Harga barang : Rp. 50.000,-**. Jika variabel kodeBarang diisi dengan karakter selain A, B, dan C maka program akan menampilkan : **Kode barang tidak valid.**

```
Masukkan Kode Barang (A, B, C) : A
Harga barang : Rp. 10.000,-
```

```
Masukkan Kode Barang (A, B, C) : B
Harga barang : Rp. 25.000,-
```

```
Masukkan Kode Barang (A, B, C) : D
Kode barang tidak valid.
```

F. PERULANGAN (LOOPING)

Perulangan atau looping dalam C++ adalah salah satu konsep pemrograman yang digunakan untuk **mengulang** atau **menjalankan suatu blok kode program secara berulang-ulang sampai suatu kondisi tertentu terpenuhi**.

1. Perulangan FOR

Perulangan FOR digunakan ketika jumlah *iterasi* (perulangan) yang diperlukan sudah diketahui sebelumnya. Bentuk umum dari perulangan for adalah sebagai berikut:

```
for (inisialisasi/nilai awal; kondisi; increment/decrement) {  
    // blok kode program yang akan diulang  
}
```

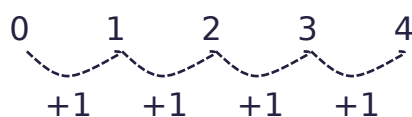
Contoh :

```
1  #include <iostream>  
2  using namespace std;  
3  int main()  
4  {  
5      string salam = "Assalamualaikum";  
6      string nama = "Rahiidah";  
7      for (int r = 0; r < 5; r++) {  
8          cout << "Halloo, " << salam << " " << nama << endl;  
9      }  
10     return 0;  
11 }
```

blok perulangan

Keterangan : **Inisialisasi** => `int r = 0`, artinya pada mulanya kita membuat variabel `r` yang berisi angka 0. **Kondisi** => `r < 5`, artinya selama nilai `r` masih kurang dari 5 maka akan dilakukan perulangan.

Increment => `r++` artinya `r + 1`. Berikut operasinya :



Nilai awal `r = 0`, karena pada proses incrementnya adalah `r + 1`, maka `r` akan terus ditambah 1 selama masih

memenuhi kondisi yaitu `r < 5`. Sehingga akan menghasilkan 5 data yang akan dilakukan untuk menjalankan bagian blok kode program yang akan diulang yaitu :

```
cout << "Halloo, " << salam << " " << nama << endl;
```

```
Halloo, Assalamualaikum Rahiidah
Halloo, Assalamualaikum Rahiidah
Halloo, Assalamualaikum Rahiidah
Halloo, Assalamualaikum Rahiidah
Halloo, Assalamualaikum Rahiidah
```

2. Perulangan WHILE

Perulangan **WHILE** digunakan ketika jumlah iterasi yang diperlukan **tidak diketahui sebelumnya**, dan **terus berlangsung selama kondisi terpenuhi**. Bentuk umum dari perulangan while adalah sebagai berikut :

```
while (kondisi) {
    // blok kode program yang akan diulang
    // perlu ada pengubahan nilai variabel yang digunakan dalam kondisi
    // untuk menghindari infinite loop (perulangan yang tak terbatas)
}
```

Contoh :

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      string salam = "Assalamualaikum";
6      string nama = "Naufal";
7      int naufal = 0;
8
9      while (naufal < 5) {
10         cout <<salam<<" " <<nama<<" " <<naufal<<endl;
11         naufal++;
12     }
13     return 0;
14 }
```

blok perulangan

Keterangan : Pada awalnya isi dari variabel naufal adalah 0, kemudian dalam perulangan WHILE, dibuat kondisi dimana selama isi dari variabel naufal masih < 5 (kurang dari 5), maka statement : `cout <<salam<<" "` `<<nama<<" "` `<<naufal<<endl;` akan terus dijalankan sampai isi dari variabel naufal sudah ≥ 5 (tidak kurang dari 5). Hal ini mengacu pada proses increment `naufal++` yang artinya adalah `naufal + 1` (naufal ditambah 1). Sehingga hasil yang ditampilkan adalah sebagai berikut :

```
Assalamualaikum Naufal 0
Assalamualaikum Naufal 1
Assalamualaikum Naufal 2
Assalamualaikum Naufal 3
Assalamualaikum Naufal 4
```

3. Perulangan DO WHILE

Perulangan DO-WHILE hampir sama dengan perulangan WHILE, hanya saja blok kode di dalamnya akan dijalankan minimal satu kali meskipun kondisinya tidak terpenuhi. Bentuk umum dari perulangan DO-WHILE adalah sebagai berikut:

```
do {
    // blok kode yang akan diulang
    // perlu ada perubahan nilai variabel yang digunakan dalam kondisi
    // untuk menghindari infinite loop
} while (kondisi);
```

Contoh :

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      string salam = "Assalamualaikum";
6      string nama = "Syafa";
7      int syafa = 0;
8
9      do {
10         cout <<salam<<" "<<nama<<" "<<syafa<<endl;
11         syafa++;
12     } while (syafa < 5);
13
14     return 0;
15 }
```

blok perulangan DO-WHILE

Keterangan : Pada awalnya isi dari variabel syafa adalah 0, kemudian dalam perulangan DO-WHILE, dijalankan 1 kali dulu statement kode program : `cout <<salam<<" "<<nama<<" "<<syafa<<endl;`

selanjutnya akan dicek dalam kondisi apakah isi dari variabel syafa masih < 5 (kurang dari 5), kalo iya perulangan akan terus dilakukan sampai kondisi syafa < 5 sudah tidak terpenuhi. Sehingga hasilnya adalah sebagai berikut :

```
Assalamualaikum Syafa 0
Assalamualaikum Syafa 1
Assalamualaikum Syafa 2
Assalamualaikum Syafa 3
Assalamualaikum Syafa 4
```

G. FUNGSI

Fungsi adalah blok kode program yang **dapat dipanggil** untuk melakukan tugas tertentu. Fungsi dapat menerima **argumen** atau **parameter** dan dapat mengembalikan nilai. Fungsi dapat digunakan untuk **memecah** program menjadi bagian-bagian yang lebih kecil dan terorganisir, sehingga lebih mudah untuk mengembangkan, memelihara, dan menguji kode. Secara umum terdapat dua jenis fungsi yaitu:

1. Fungsi Standar (Built-in Functions):

Fungsi yang sudah disediakan oleh bahasa C++ itu sendiri dan dapat digunakan tanpa perlu mendefinisikan ulang. Beberapa contoh fungsi standar di antaranya adalah **main()**, **printf()**, **scanf()**, dan **sqrt()**.

2. Fungsi Pengguna (User-defined Functions):

Fungsi yang kita buat sendiri untuk melakukan tugas tertentu. Format dasarnya :

```
tipe_data nama_fungsi(parameter){
    // statement blok kode program
    return nilai; }
```

Keterangan :

- **tipe_data** adalah tipe data yang dikembalikan oleh fungsi, seperti int, float, string.

- **nama_fungsi** adalah nama yang diberikan pada fungsi.
- **parameter** adalah daftar parameter atau argumen yang diperlukan oleh fungsi, bisa berupa satu atau lebih parameter, atau tidak ada parameter sama sekali.
- **blok kode** adalah bagian yang berisi pernyataan yang dieksekusi oleh fungsi.
- **return nilai** adalah pernyataan yang mengembalikan nilai dari fungsi tersebut.

membuat fungsi dengan nama

Contoh :

```

1  #include <iostream>
2  using namespace std;
3
4  int perkalian(int A, int B){
5      int hasil = A * B;
6      return hasil;
7  }
8
9  int main(){
10     int R = 4, N = 6;
11     int S = perkalian(R, N);
12     cout << "Hasil perkalian R dan N adalah " << S << endl;
13     return 0;
14 }

```

parameter

nilai kembalian ketika fungsi

Memanggil fungsi perkalian()

fungsi main()

Hasil perkalian R dan N adalah 24

Dalam contoh di atas, **fungsi perkalian()** yang dibuat digunakan untuk mengalikan dua bilangan bulat dan mengembalikan hasilnya. Nilai kembalian kemudian digunakan di dalam **fungsi main()** untuk tampilkan di layar.

Contoh fungsi **tanpa** nilai pengembalian :

membuat 3 buah fungsi tanpa nilai

```

1  #include <iostream>
2  using namespace std;
3
4  void sapaRahiidah() {
5      cout << "Selamat Pagi Rahiidah" << endl;
6  }
7
8  void sapaNaufal() {
9      cout << "Selamat Siang Naufal" << endl;
10 }
11
12 void sapaSyafa() {
13     cout << "Selamat Sore Syafa" << endl;
14 }
15
16 int main()
17 {
18     sapaRahiidah();
19     sapaNaufal();
20     sapaSyafa();
21
22     return 0;
23 }

```

Diagram illustrating function calls within the `main()` function. A red box highlights the function definitions, and another red box highlights the function calls in `main()`. A dashed box with an arrow points to the calls, stating: *memanggil ketiga fungsi tersebut di dalam fungsi main()*.

Hasil yang ditampilkan di layar ketika program dijalankan :

```

Selamat Pagi Rahiidah
Selamat Siang Naufal
Selamat Sore Syafa

```

3. Parameter dan Argumen di dalam Fungsi

- **Parameter** adalah variabel yang didefinisikan dalam **header fungsi**, yang digunakan untuk menerima input dari pemanggil fungsi. Parameter ini digunakan di dalam fungsi untuk **melakukan operasi tertentu** dan **menghasilkan output**.
- **Argumen** adalah nilai yang diberikan ke fungsi **saat dipanggil**. Argumen digunakan untuk **memberikan nilai ke parameter** yang terdapat pada fungsi.

Format dasar fungsi dengan parameter dan argumen :

```

1  #include <iostream>
2
3  using namespace std;
4
5  tipeDataKembalian namaFunction(tipeData param1, tipeData param2) {
6      // Isi function disini...
7      // Isi function disini...
8      return nilai;
9  }
10
11 int main()
12 {
13     // Jalankan function
14     namaFunction(arg1, arg2)
15
16     return 0;
17 }

```

Contoh Fungsi dengan 1 parameter dan 1 argumen dengan nilai kembalian :

```

1  #include <iostream>
2  using namespace std;
3
4  //fungsi dengan satu parameter
5  int pangkat_dua(int parameter){
6      int hasil = parameter * parameter;
7      return hasil;
8  }
9
10 int main() {
11     int argumen = 5;
12     // memanggil fungsi pangkat dua dengan argumen 5
13     int pangkat = pangkat_dua(argumen);
14     cout<< "Hasil pangkat 2 dari "<<argumen<<"=" <<pangkat<< endl;
15     return 0;
16 }

```

Diagram annotations:

- A dashed box labeled "parameter" points to the `int parameter` in the function signature of `pangkat_dua`.
- A dashed box labeled "argumen" points to the `argumen` argument in the function call `pangkat_dua(argumen)` inside the `main` function.

Hasil yang ditampilkan :

```

Hasil pangkat 2 dari 5=25

```

Pada contoh di atas, fungsi `pangkat_dua` memiliki satu parameter "**parameter**" yang digunakan untuk melakukan operasi perpangkatan. Pada fungsi `main`, kita memanggil fungsi `pangkat_dua` dengan argumen "**argumen**" yang bernilai 5. Selanjutnya, hasil perpangkatan dikembalikan dan disimpan pada variabel "**pangkat**", kemudian ditampilkan ke layar menggunakan fungsi **cout**.

Contoh Fungsi dengan 2 parameter dan 2 argumen tanpa nilai kembalian :

```
1  #include <iostream>
2
3  using namespace std;
4
5  void cariLuasSegitiga(int alas, int tinggi) {
6      float luasSegitiga = (alas * tinggi) * 0.5;
7      cout << "Luas Segitiga adalah : " << luasSegitiga << endl;
8  }
9
10 int main()
11 {
12     cariLuasSegitiga(10, 8);
13     cariLuasSegitiga(15, 13);
14     cariLuasSegitiga(55, 37);
15
16     return 0;
17 }
```

parameter

argumen

Hasilnya adalah :

```
Luas Segitiga adalah: 40
Luas Segitiga adalah: 97.5
Luas Segitiga adalah: 1017.5
```

Pada baris ke-5, kita membuat fungsi **cariLuasSegitiga()** yang definisikan dengan 2 buah parameter bertipe **int**, yakni **alas** dan **tinggi**. Penamaan parameter boleh bebas, sama mengikuti aturan penamaan variabel / identifier, misalnya tidak boleh ada spasi dan tidak boleh diawali angka. Pada baris 12 - 14 kita menjalankan fungsi **cariLuasSegitiga()** sebanyak 3 kali dengan nilai argumen yang berbeda-beda. Sebagai contoh, ketika compiler C++ memproses perintah **cariLuasSegitiga(10, 8)**, angka 10 ini akan mengisi parameter **alas**, dan angka 8 akan mengisi parameter **tinggi**. Selanjutnya kedua parameter masuk ke perhitungan luas di baris 6. Hasilnya kemudian di tampilkan dari perintah **cout << "Luas Segitiga adalah: " << luas << endl;**.

4. Perintah Return di dalam Fungsi

Perintah **return** di C++ digunakan untuk mengembalikan nilai dari suatu fungsi dan mengakhiri eksekusi dari suatu fungsi. Pernyataan **return** terdiri dari kata kunci "return" diikuti oleh nilai yang ingin dikembalikan atau tanpa nilai (**void**). Umumnya, sintaks dari perintah "return" pada bahasa pemrograman C++ adalah sebagai berikut :

```
return nilai;
```

Di mana "**nilai**" adalah nilai yang akan **dikembalikan** oleh fungsi tersebut. Jika fungsi tidak mengembalikan nilai, perintah "return" **dapat dihilangkan**, atau "return" dapat **digunakan tanpa nilai**. (return 0). Sebagai contoh, fungsi berikut ini menghitung jumlah dari dua bilangan bulat dan mengembalikan hasilnya :

```
int jumlah(int a, int b) {  
    int hasil = a + b;  
    return hasil;  
}
```

Pada contoh ini, perintah "**return**" akan mengembalikan nilai dari variabel "hasil" ke fungsi pemanggil. Fungsi pemanggil kemudian dapat menggunakan nilai yang dikembalikan tersebut sesuai dengan kebutuhannya.

5. Fungsi Rekursif

Fungsi **REKURSIF** adalah fungsi yang **memanggil dirinya sendiri**. **Pemanggilan diri ini terjadi secara berulang hingga suatu kondisi berhenti terpenuhi**. Fungsi rekursif biasanya digunakan untuk menyelesaikan masalah yang dapat dipecahkan menjadi sub-masalah yang lebih kecil. Berikut format dasar dari fungsi REKURSIF :

```
5 tipe_data nama_fungsi(parameter) {  
6     if (kondisi basis) {  
7         // kasus dasar  
8         return nilai;  
9     }  
10    else {  
11        // rekursi  
12        return operasi(nama_fungsi(argument));  
13    }  
14 }
```

Keterangan :

- **tipe_data**: jenis data yang dihasilkan oleh fungsi (misalnya int, double, bool, dll.)
- **nama_fungsi**: nama fungsi yang akan digunakan
- **parameter**: variabel yang akan digunakan sebagai input dari fungsi
- **kondisi basis**: **kondisi yang akan menentukan kapan rekursi dihentikan**

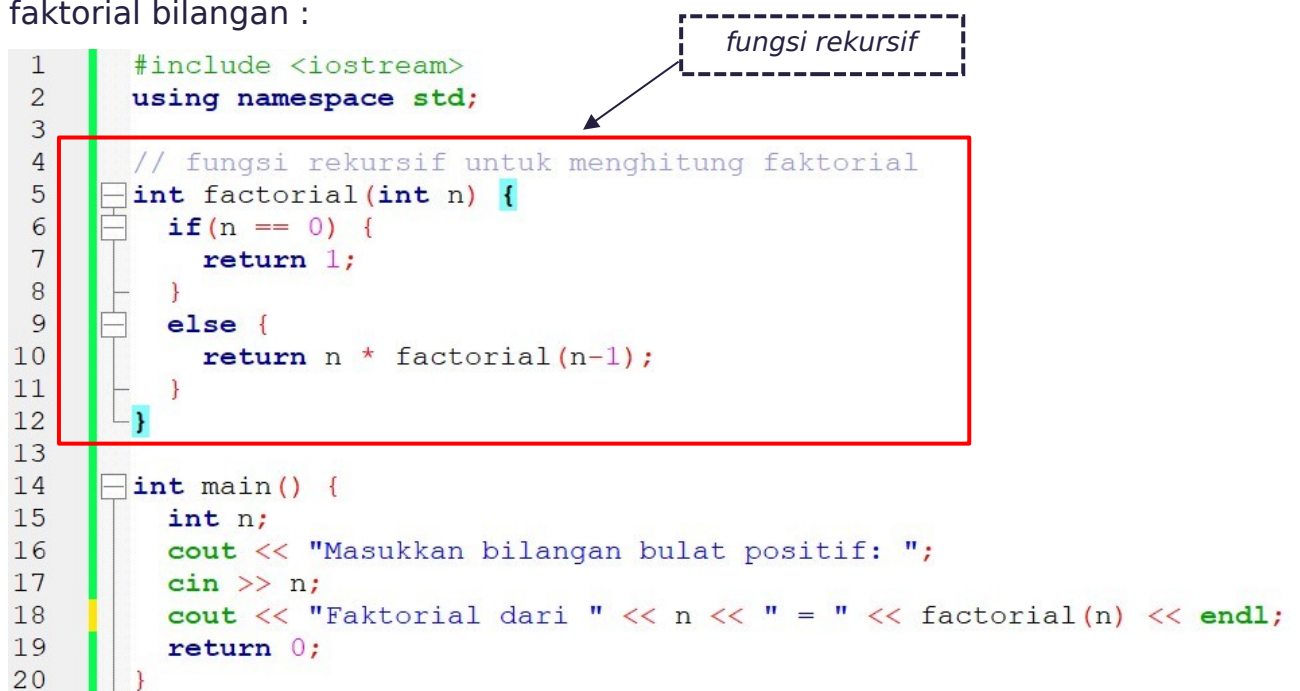
- **kasus dasar:** operasi yang dilakukan ketika kondisi basis terpenuhi
- **rekursi:** pemanggilan kembali fungsi dengan argumen yang berbeda
- **operasi:** operasi yang dilakukan pada nilai yang dikembalikan dari fungsi rekursif

Perlu diperhatikan bahwa dalam sebuah fungsi rekursif, **kondisi basis haruslah terpenuhi untuk mencegah terjadinya infinite loop**. Selain itu, perlu juga memperhatikan bahwa penggunaan fungsi rekursif dapat memakan memori yang cukup besar, terutama pada kasus dengan jumlah iterasi yang besar.

Contoh sederhana dari fungsi rekursif adalah fungsi untuk menghitung **faktorial** suatu bilangan. Faktorial dari bilangan n didefinisikan sebagai hasil kali dari bilangan bulat positif dari 1 hingga n . Faktorial dari n ditulis sebagai $n!$. Sehingga:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$$

Berikut adalah contoh implementasi fungsi rekursif untuk menghitung faktorial bilangan :



```
1  #include <iostream>
2  using namespace std;
3
4  // fungsi rekursif untuk menghitung faktorial
5  int factorial(int n) {
6      if(n == 0) {
7          return 1;
8      }
9      else {
10         return n * factorial(n-1);
11     }
12 }
13
14 int main() {
15     int n;
16     cout << "Masukkan bilangan bulat positif: ";
17     cin >> n;
18     cout << "Faktorial dari " << n << " = " << factorial(n) << endl;
19     return 0;
20 }
```

Tampilan program jika diinputkan bilangan 5.

```
Masukkan bilangan bulat positif: 5
Faktorial dari 5 = 120
```

Fungsi factorial di atas digunakan untuk menghitung nilai faktorial dari suatu bilangan bulat. Fungsi ini akan memanggil dirinya sendiri secara rekursif dengan nilai n-1 hingga n mencapai nilai 0, pada saat itu, fungsi akan mengembalikan nilai 1.

Contoh di atas dapat dijalankan dengan memberikan input berupa bilangan bulat positif yang ingin dihitung faktorialnya. Program akan mengeluarkan output berupa nilai faktorial dari bilangan tersebut.

“Don’t Count the Days, Make the Days Count”
Thank You!

BAB 1 GERBANG LOGIKA

A. Tabel Kebenaran

AND

INPUT		OUTPUT T
A	B	
0	0	0
1	0	0
0	1	0
1	1	1

OR

INPUT		OUTPUT T
A	B	
0	0	0
1	0	1
0	1	1
1	1	1

XOR

INPUT		OUTPUT T
A	B	
0	0	0
1	0	1
0	1	1
1	1	0

NAND

INPUT		OUTPUT T
A	B	
0	0	1
1	0	1
0	1	1
1	1	0

NOR

INPUT		OUTPUT T
A	B	
0	0	1
1	0	0
0	1	0
1	1	0

XNOR

INPUT		OUTPUT T
A	B	
0	0	1
1	0	0
0	1	0
1	1	1

NOT

INPUT	OUTPUT T
-------	-------------

0	1
1	0

B. Contoh Soal !

- Perhatikan operasi logika berikut!

$P = ((\text{not } A) \text{ and } B) \text{ or } (((\text{not } C) \text{ or } D) \text{ and } E)$

$Q = ((\text{not } A) \text{ or } B) \text{ and } (((\text{not } C) \text{ and } (\text{not } D)) \text{ or } (\text{not } E))$

$R = P \text{ and } Q$

Jika nilai $A = \text{True}$, $B = \text{True}$, $C = \text{True}$, $D = \text{True}$, dan $E = \text{False}$, tentukan nilai P , Q dan R berturut-turut?

A. $P=\text{True}$, $Q= \text{True}$, $R= \text{True}$

B. $P=\text{False}$, $Q= \text{True}$, $R= \text{True}$

C. $P=\text{False}$, $Q= \text{True}$, $R= \text{False}$

D. $P=\text{False}$, $Q= \text{False}$, $R= \text{False}$

E. $P=\text{True}$, $Q= \text{False}$, $R= \text{False}$

- Perhatikan operasi logika berikut!

$P = (A \text{ and } B) \text{ or } (C \text{ and } (\text{not } D))$

$Q = (A \text{ or } C) \text{ and } (B \text{ or } (\text{not } D))$

$R = P \text{ and } Q$

Jika nilai $A = \text{True}$, $B = \text{False}$, $C = \text{True}$, dan $D = \text{False}$, tentukan nilai P , Q , dan R berturut-turut?

A. $P=\text{False}$, $Q= \text{True}$, $R= \text{False}$

B. $P=\text{True}$, $Q= \text{False}$, $R= \text{False}$

C. $P=\text{False}$, $Q= \text{False}$, $R= \text{False}$

D. $P=\text{True}$, $Q= \text{True}$, $R= \text{True}$

E. $P=\text{True}$, $Q= \text{True}$, $R= \text{False}$

- Pak Dengklek mempunyai tiga variabel boolean P , Q , dan R . Pak Dengklek ingin membuat sebuah operasi logika yang mengembalikan nilai TRUE jika dan hanya jika minimal dua variabel dari P , Q , atau R mempunyai nilai TRUE . Operasi logika mana yang memenuhi kriteria Pak Dengklek tersebut?

A. $(P \text{ or } Q \text{ or } R) \text{ and } (\text{not } P \text{ or } \text{not } Q \text{ or } \text{not } R)$

B. $(\text{not } (P \text{ or } Q)) \text{ or } (\text{not } (Q \text{ or } R)) \text{ or } (\text{not } (R \text{ or } P))$

- C. (not (P and Q)) and (not (Q and R)) and (not (R and P))
- D. (P and Q) or (Q and R) or (R and P)
- E. (P or Q) and (Q or R) and (R or P)

P = TRUE
Q = TRUE
R = FALSE

4. Pak Dengklek memiliki tiga variabel boolean P, Q, dan R. Pak Dengklek ingin membuat sebuah operasi logika yang mengembalikan nilai TRUE jika hanya satu variabel dari P, Q, atau R yang mempunyai nilai TRUE. Operasi logika **mana yang tidak memenuhi kriteria** Pak Dengklek tersebut?
- A. (P or Q or R) and (not P or not Q or not R) -> TRUE
 - B. (not (P or Q)) or (not (Q or R)) or (not (R or P)) -> TRUE
 - C. (not (P and Q)) and (not (Q and R)) and (not (R and P)) -> TRUE
 - D. (P and Q) or (Q and R) or (R and P) -> false
 - E. (P or Q) and (Q or R) and (R or P) -> false

P = TRUE
Q = FALSE
R = FALSE

PERTEMUAN KE-2

KOMBINASI

A. Konsep Dasar

Kombinasi adalah salah satu konsep dalam matematika yang digunakan untuk menghitung **berapa banyak cara yang berbeda** untuk memilih sejumlah *robjek* dari sekelompok *n* objek yang ada, **tanpa memperhatikan urutan**. Ini diwakili oleh simbol nCr dan dihitung dengan rumus berikut :

$$nCr = \frac{n!}{(n-r)!r!}$$

Keterangan:

nCr : jumlah kombinasi yang mungkin.
n : adalah jumlah total data yang tersedia.
r : adalah jumlah data yang akan dipilih.

n! (n faktorial) : adalah hasil perkalian semua bilangan bulat positif dari 1 hingga n

Contoh :

Terdapat 5 anak (sebut saja A, B, C, D, dan E) yang mana akan dipilih 3 anak untuk mengikuti kompetisi. Berapa banyak susunan tim berbeda yang dapat dibentuk?

Susunan ABC, ACB, BAC, BCA, CAB, CBA merupakan 1 susunan yang sama. Jadi keenam susunan tersebut dianggap sebagai 1 susunan. Jadi berapa banyak **cara memilih** susunan tim dari kasus diatas adalah :

$$nCr = \frac{5!}{(5-3)! \cdot 3!} = \frac{5 \times 4 \times 3 \times 2 \times 1}{2 \times 1 \times 3 \times 2 \times 1} = 10 \quad \boxed{}$$

Seorang siswa memiliki 8 buah buku matematika, 5 buku fisika, dan 4 buku kimia. Berapa banyak cara yang berbeda untuk memilih 2 buku matematika, 2 buku fisika, dan 1 buku kimia?

Kita akan menghitung kombinasi masing-masing jenis buku terlebih dahulu:

$$\text{Kombinasi buku matematika} = {}^8nCr = \frac{8!}{(8-2)! \cdot 2!} = 28$$

$$\text{Kombinasi buku fisika} = {}^5nCr = \frac{5!}{(5-2)! \cdot 2!} = 10$$

$$\text{Kombinasi buku kimia} = {}^4nCr = \frac{4!}{(4-1)! \cdot 1!} = 4$$

Karena kita ingin menghitung berapa banyak cara yang berbeda untuk memilih buku-buku tersebut **secara bersamaan**, kita dapat **mengalikan** jumlah cara-cara yang berbeda untuk setiap jenis buku.

$$\text{Total Cara} = 28 \times 10 \times 4 = 1120$$

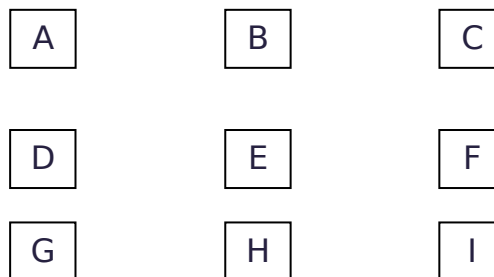
B. Contoh Soal !

1. Tahun ini Pak Dengklek ditunjuk menjadi ketua panitia Olimpiade Internasional Bebek (OIB). Untuk memberikan pengalaman kepada bebek-bebeknya, Pak Dengklek berencana memilih 10 dari 15 bebek yang dimilikinya untuk menjadi peserta. Tentunya kita tahu bahwa di antara 15 bebek tersebut, ada empat bebek kesayangan Pak Dengklek, yaitu Kwak, Kwik, Kwek dan Kwok. Kwak dan Kwik harus dipilih untuk menjadi peserta lomba karena keduanya yang paling pintar. Sedangkan Kwek dan Kwok tidak bisa dipilih sebab saat ini sedang sakit. Ada berapa banyak cara memilih bebek-bebek sebagai peserta OIB?

2. Karena merasa bebeknya sudah terlalu banyak, Pak Dengklek ingin mengurangi jumlah bebeknya dengan cara menjual semua bebek yang tidak 'unggul'. Pak Dengklek menyiapkan 10 soal dari nomor 1 sampai 10 untuk dikerjakan semua bebeknya (benar bernilai 1, salah bernilai 0). Bebek dikatakan 'unggul' jika mendapat nilai paling sedikit 8. Kwak adalah bebek yang sangat cerdas dan telah mengetahui jawaban yang benar dalam menjawab soal apapun. Namun, Kwak tidak selalu ingin memberikan jawaban yang benar. Agar Kwak tidak dijual, berapa banyaknya cara Kwak memilih soal yang akan dijawab dengan benar jika Kwak menjawab soal nomor 1, 3, dan 5 dengan benar?
3. Jika diberikan sembilan buah patok pada lahan Pak Dengklek sebagai berikut:



Pak Dengklek ingin membuat sebuah kandang yang berbentuk segitiga dimana setiap pojok sudut kandang harus merupakan patok-patok tersebut. Sisi kandang boleh melewati atau mengandung patok-patok lainnya. Ada berapa banyak kemungkinan kandang yang bisa dibangun oleh Pak Dengklek?



PERTEMUAN KE-3

PERMUTASI

A. Konsep Dasar

Permutasi adalah salah satu konsep dalam matematika yang digunakan untuk menghitung **berapa banyak cara yang berbeda** untuk memilih sejumlah objek dari sekelompok n objek yang ada, **dengan memperhatikan urutan**. Ini diwakili oleh simbol nPr dan dihitung dengan rumus berikut :

$${}_nP_r = \frac{n!}{(n-r)!}$$

Keterangan:

nPr : jumlah permutasi yang mungkin.

n : adalah jumlah total data yang tersedia.

r : adalah jumlah data yang akan dipilih.

$n!$ (n faktorial) : adalah hasil perkalian semua bilangan bulat positif dari 1 hingga n

Contoh :

Anda memiliki 5 buku berbeda dan ingin mengatur 3 di antaranya dalam urutan tertentu di rak buku Anda. Berapa banyak cara yang mungkin untuk melakukannya?

$${}_5P_3 = \frac{5!}{(5-3)!} = \frac{5 \times 4 \times 3 \times 2 \times 1}{2 \times 1} = \frac{120}{2} = 60$$

Dalam sebuah kelompok 5 siswa, yaitu A, B, C, D, dan E, mereka akan duduk bersebelahan di kursi di sekitar meja bundar. Berapa banyak cara yang mungkin untuk mengatur mereka?

$${}_5P_r = \frac{5!}{(5-5)!} = \frac{5 \times 4 \times 3 \times 2 \times 1}{0!} = \frac{120}{1} = 120$$

B. Contoh Soal !

1. Pak Dengklek beserta ketiga bebeknya yaitu kwak, kwik dan kwek memesan tiket kereta untuk pergi ke Surabaya. Ada berapa cara bagi Pak Dengklek dan ketiga bebeknya tersebut untuk menempati tempat duduk yang dipesan?
2. Dalam perkumpulan komunitas bebek, akan dipilih ketua dan wakil ketua. Terdapat 7 ekor bebek sebagai kandidat yang dapat dipilih untuk berada di posisi tersebut. Berapa pasangan yang mungkin untuk terpilih sebagai pengurus inti dalam komunitas bebek tersebut?
3. Sebuah tim sepak bola menggunakan formasi dengan 1 penyerang. Bila semua pemain dapat mengisi semua posisi, berapa jumlah alternatif penempatan pemain untuk posisi penyerang serta penjaga gawang jika pemain cadangan tidak dianggap?
4. Sebuah organisasi PKK akan memilih ketua, sekretaris, dan bendahara dari 7 pengurus PKK tersebut. Bila pengurus A telah dipastikan akan

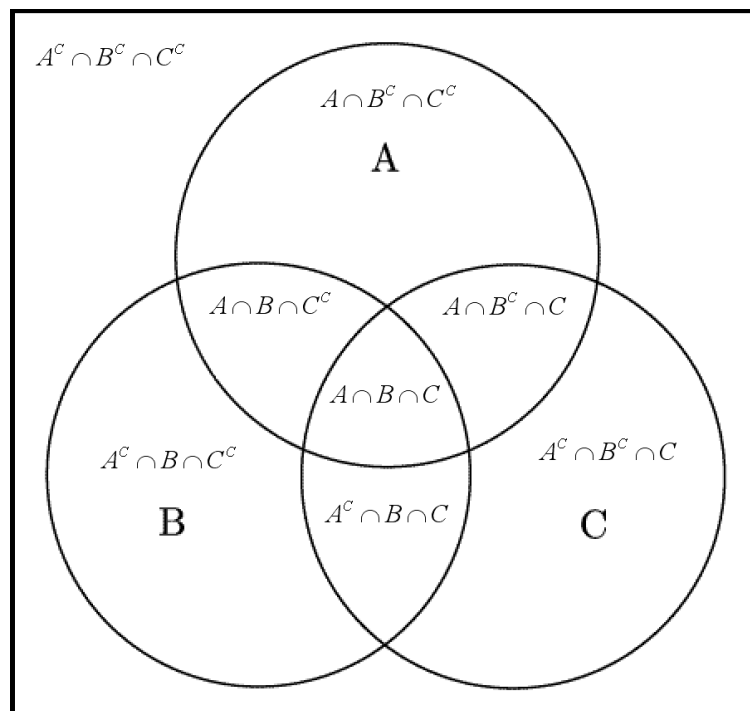
terpilih pada salah satu posisi, berapa banyak kemungkinan pengurus yang dapat terbentuk?

PERTEMUAN KE-4

HIMPUNAN

A. Konsep Dasar

Himpunan adalah konsep matematika yang digunakan untuk mengelompokkan objek-objek dengan karakteristik atau sifat yang sama ke dalam satu entitas tunggal.



B. Contoh Soal !

1. Pak Dengklek akan mengadakan tasyakuran atas kemenangan Kwak, Kwik, Kwek, Kwok dan Kwuk dalam mengikuti Olimpiade Sains Nasional Bebek (OSNB). Tasyakuran ini rencananya akan mengundang 192 tamu undangan. Dalam rangka menyiapkan jamuan yang sesuai untuk para tamu, pak Dengklek telah mensurvei preferensi setiap tamunya terhadap 3 jenis alternatif lauk, yaitu: bebek goreng, bebek rebus atau bebek bakar. Hasil survei menunjukkan: 67 orang suka makan bebek goreng, 73 orang suka makan bebek rebus, 51 orang suka bebek bakar, 11 orang suka bebek goreng dan bebek rebus, 9 orang suka bebek rebus dan bebek bakar, 12 orang suka bebek goreng dan bebek bakar, dan 5 orang yang suka ketiga-tiganya. Jawablah pertanyaan berikut !
 - a. Berapakah orang yang tidak suka ketiga jenis lauk tersebut?
 - b. Berapakah orang yang hanya menyukai bebek goreng?

- c. Berapakah orang yang hanya menyukai bebek rebus?
 - d. Berapakah orang yang hanya menyukai bebek bakar?
 - e. Berapakah orang yang hanya menyukai bebek goreng dan bebek rebus?
 - f. Berapakah orang yang hanya menyukai bebek rebus dan bebek bakar?
 - g. Berapakah orang yang hanya menyukai bebek goreng dan bebek bakar?
2. Dalam pemilihan Ketua Asosiasi Bebek, terdapat sekitar 120 bebek yang akan melakukan voting dengan tiga kandidat ketua yaitu Kwak, Kwik, dan Kwek. Dalam pemilihan ini, setiap bebek boleh tidak melakukan voting atau melakukan voting ke lebih dari satu kandidat. Diketahui bahwa ada 65 bebek yang memilih Kwak, 45 bebek memilih Kwik, dan 42 bebek yang memilih Kwek. Kemudian diketahui juga bahwa ada 20 bebek yang memilih Kwak dan Kwik sekaligus, 25 bebek memilih Kwak dan Kwek sekaligus, dan 15 bebek yang memilih Kwik dan Kwek sekaligus. Secara aturan jika bebek memilih hanya satu kandidat maka seluruh suara bulat (nilai 3) akan diberikan kepada kandidat tersebut, jika bebek memilih lebih dari satu kandidat maka suara bulat (nilai 3) akan dibagi rata sebanyak kandidat yang dipilih. Kandidat yang mendapatkan suara terbesar pertama, kedua dan ketiga berturut-turut akan menjadi Ketua, Wakil Ketua dan Sekretaris. Jawablah pertanyaan berikut !
- a. Siapa yang akan menjadi ketua dan berapa perolehan skornya ?
 - b. Siapa yang akan menjadi wakil ketua dan berapa perolehan skornya ?
 - c. Siapa yang akan menjadi sekretaris dan berapa perolehan skornya ?

